



Diamond Hands Vault Security Review

May 13, 2026

Contents

1. About SBSecurity	3
2. Disclaimer	3
3. Risk classification	3
4. Executive Summary	4
5. Findings	5
Low severity	6
[L-01] notifyAllTimeHigh unlocks when price equals the target, not only when it exceeds it	6
Informational severity	7
[I-01] stETH dust may accumulate in the vault on every deposit	7
[I-02] notifyAllTimeHigh return value is redundant with the emitted event	8

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Twitter [@Slavcheww](https://twitter.com/Slavcheww).

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security-related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1. Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2. Likelihood

- **High** - almost **certain** to happen, easy to perform, or highly incentivized.
- **Medium** - only **conditionally possible**, but still relatively likely.
- **Low** - requires specific state or **little-to-no incentive**.

3.3. Action required for severity levels

- High - **Must** fix (before deployment if not already deployed).
- Medium - **Should** fix.
- Low - **Could** fix.



4. Executive Summary

A time-boxed security review of the **sstrenev/diamond-hands-vault** repository was conducted by **SBSecurity** who collectively identified **3 issues** in total.

Overview

Project	Diamond Hands Vault
Commit Hash	4b0be34e6448bc457c2ca1ba8613a1aeade15ed8
Resolution	-
Timeline	May 13, 2026

Scope

src/DiamondHandsVault.sol

Post Audit Condition

The codebase demonstrates solid development practices. Additional audits would be a bonus, but are not mandatory.

5. Findings

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Informational Risk	2

ID	Title	Severity	Resolution
[L-01]	<code>notifyAllTimeHigh</code> unlocks when price equals the target, not only when it exceeds it	Low	Acknowledged
[I-01]	<code>stETH</code> dust may accumulate in the vault on every deposit	Info	Acknowledged
[I-02]	<code>notifyAllTimeHigh</code> return value is redundant with the emitted event	Info	Acknowledged

Low severity

[L-01] `notifyAllTimeHigh` unlocks when price equals the target, not only when it exceeds it

Severity: Low Risk

Description: The check uses `<` instead of `<=`, so `notifyAllTimeHigh` succeeds when `currentPrice == allTimeHigh`. The name implies a new high, which requires the price to strictly exceed the previous one.

[src/DiamondHandsVault.sol#L166](#)

```
if (uint256(currentPrice) < allTimeHigh) revert AllTimeHighNotReached();
```

Impact:

Anyone can permanently unlock the vault as soon as Chainlink reports a price equal to `allTimeHigh`, without ETH ever printing a new high. Deposits get disabled and the early-exit fee stops earlier than the design intends.

Recommendation: In the `request_fast_redeem_status` function make the following changes:

```
- if (uint256(currentPrice) < allTimeHigh) revert AllTimeHighNotReached();  
+ if (uint256(currentPrice) <= allTimeHigh) revert AllTimeHighNotReached();
```

Resolution: Acknowledged

Informational severity

[I-01] stETH dust may accumulate in the vault on every deposit

Severity: Information Risk

Description: `deposit` submits ETH to Lido, reads back the resulting share amount, converts it to a stETH balance, and wraps that balance into wstETH:

```
uint256 shares = IStETH(stETH).submit{ value: msg.value }(address(this));
uint256 stETHAmount = IStETH(stETH).getPooledEthByShares(shares);

IERC20(stETH).approve(wstETH, stETHAmount);

uint256 wstETHAmount = IWstETH(wstETH).wrap(stETHAmount);
```

Both `getPooledEthByShares` and the stETH share/balance conversions inside `transferFrom` floor-divide, which is the well known stETH 1-2 wei rounding behaviour. The vault may end up holding a tiny amount of unwrapped stETH after each deposit.

Impact:

Not exploitable. The dust is permanently stuck in the vault since there is no sweep function, and it grows by a constant tiny amount per deposit.

Recommendation:

Consider adding a permissioned `sweepStETH` to forward dust to `exitFeeRecipient`.

Resolution: Acknowledged

[I-02] `notifyAllTimeHigh` return value is redundant with the emitted event

Severity: Information Risk

Description: `notifyAllTimeHigh` returns `newAllTimeHigh` and also emits it in `ETHAllTimeHighReached`:

[src/DiamondHandsVault.sol#L161-L174](#)

```
function notifyAllTimeHigh() external returns (uint256 newAllTimeHigh) {
    if (allTimeHighReached) revert AllTimeHighReached();

    (int256 currentPrice,) = getETHPrice();

    if (uint256(currentPrice) < allTimeHigh) revert AllTimeHighNotReached();

    newAllTimeHigh = uint256(currentPrice);

    allTimeHighReached = true;

    emit ETHAllTimeHighReached(newAllTimeHigh);
}
```

The function is only meaningful when called as a transaction (it mutates state), so the return value is not observable to off chain callers, while indexers already get the same value through the event.

Impact:

No security impact. Slightly wider ABI surface and a misleading signature that suggests off chain consumers should read the return.

Recommendation:

Drop the return value:

```
- function notifyAllTimeHigh() external returns (uint256 newAllTimeHigh) {
+ function notifyAllTimeHigh() external {
    if (allTimeHighReached) revert AllTimeHighReached();

    (int256 currentPrice,) = getETHPrice();

    if (uint256(currentPrice) < allTimeHigh) revert AllTimeHighNotReached();

-     newAllTimeHigh = uint256(currentPrice);
-
    allTimeHighReached = true;

-     emit ETHAllTimeHighReached(newAllTimeHigh);
+     emit ETHAllTimeHighReached(uint256(currentPrice));
}
```

Resolution: Acknowledged